

BrokerVerse OOTB

Insurance broking platform by iorta TechNXT

Solution Architecture · Document 09 of 11

Backup and Recovery

Product	BrokerVerse OOTB
Document	09 Backup and Recovery
Version	1.2
Date	04 October 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse OOTB Solution Architecture: 09 Backup and Recovery
Document ID	BV-SA-09
Version	1.2
Status	Issued for review
Date	04 October 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, deploy/, docker-compose.yml, .github/workflows, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue
1.1	29 September 2026	iorta TechNXT	Updated to the current system: placement journey, co-insurance, posting rules, period end and BIR tax, bank reconciliation, document numbering, commission rate matrix, single PDF engine, uploads and go-live imports, housekeeping, business time zone, broker roles, deployment on the BrokerVerse URL
1.2	04 October 2026	iorta TechNXT	New modules (AML/CFT, BIR, payables and fixed assets, distribution, fleet and marine, My Work, go-live workbench), integration framework, encryption and masking of personal data, branding and e-signatures, release pipeline; database at migration 0322 (reference data), new ER diagrams

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

- 1 Introduction..... 5
 - 1.1 Purpose and scope..... 5
- 2 What to back up..... 6
- 3 Backup design..... 8
 - 3.1 AWS production deployment..... 9
 - 3.2 Docker Compose and single-server installations..... 9
 - 3.3 Consistency between database and files..... 9
- 4 Restore procedures..... 10
 - 4.1 Database: point-in-time restore (AWS)..... 10
 - 4.2 Database: restore from snapshot or logical dump..... 10
 - 4.3 Files (EFS)..... 10
 - 4.4 Front end..... 11
 - 4.5 Secrets..... 11
 - 4.6 Application release rollback..... 11
- 5 Restore testing schedule..... 12
- 6 Related documents..... 13
- 7 Glossary..... 14

1 Introduction

1.1 Purpose and scope

This document defines what must be backed up for BrokerVerse OOTB, how (in the AWS production deployment and in a Docker Compose installation), how long backups are kept, and the step-by-step procedures to restore the database, the files, the front end and the secrets, together with the restore test schedule. The objectives the procedures must meet are in document 08.

Recommended / to be confirmed by the business and DevOps: The code contains no backup automation. `deploy/README.md` requires daily automated database snapshots with at least 30 days' retention, a backup of the upload volume and of `DATA_ENCRYPTION_KEY`, and one restore test before go-live. Retention periods, schedules and tools beyond that are recommendations.

2 What to back up

Asset	Location	Why it matters
Database	PostgreSQL database <code>brokerverse</code> (for example Amazon RDS)	All business, financial, configuration, security and audit data (250 tables; 260 with packages B and G). The only copy of transactions.
Uploaded and generated files	<code>UPLOAD_DIR</code> (<code>/app/uploads</code> ; EFS in the target)	Policy documents, IDs, vehicle photos, claim documents, signatures, logos, statements and generated reports. The <code>documents</code> table holds only the keys, not the content.
<code>DATA_ENCRYPTION_KEY</code>	Secrets Manager / environment	Decrypts the two-factor (TOTP) secrets stored in <code>users</code> and keys the reset-code hashes. A database restored without the same key works, but every user with two-factor sign-in must be reset by an administrator and enrol again.
<code>PII_ENCRYPTION_KEY</code> (package B)	Secrets Manager / environment	Decrypts the TIN, government ID and bank account numbers stored encrypted in the database (and in every backup). A backup cannot be read without the key it was written with: keep every key version, including the previous key after a rotation, as long as backups taken with it are kept.
<code>JWT_SECRET</code>	Secrets Manager / environment	Signs sessions, signed file links and approval / download tokens. Losing it signs everyone out and invalidates links already sent (for example quotation approval e-mails); no data is lost.
Other configuration	<code>DATABASE_URL</code> , <code>SMTP_URL</code> , <code>CORS_ORIGINS</code> , <code>PUBLIC_BASE_URL</code> , <code>ADMIN_PASSWORD</code> , task definitions	Needed to recreate the environment.
Front-end build	S3 bucket behind CloudFront	Rebuilt from Git by the pipeline; S3 versioning allows an immediate rollback.
Code and schema	Git repositories (front end, backend incl. <code>db/migrations</code> , <code>db/seeds</code> , <code>seeds/settings.json</code> , <code>seeds/jobs.json</code>)	The schema and reference data are recreated from the code; backups of Git hosting are the responsibility of the repository owner.
Container images	Amazon ECR	Allow redeploying exactly the running release; keep at least the last 10 release tags.

Application settings, masters, product templates, users and roles are data in the database (edited in the application), so they are covered by the database backup. The Go-Live Data Workbench exports the configuration as a workbook (Master > Go-Live and Data > Go-Live Data Load, configuration kit with current data) and compares two environments (`backend/scripts/compare-environments.js`). The release pipeline takes a verified `pg_dump` before every deployment to Pre-Prod and Production (`deploy/ec2/release.sh backup`, newest ten kept on the server, copy to `BACKUP_S3_URI`); these are restore points for a release, not the backup plan.

Recommended / to be confirmed by the business and DevOps: Take a configuration export before every release and before large configuration changes, for audit and for comparison between environments: `pg_dump --data-only --table=app_settings --table=master_types --table=master_records --table=product_templates --table=product_components --table=product_risk_mappings --table=product_risk_sections --table=report_definitions --table=scheduled_jobs --table=roles --table=permissions --table=role_permissions --table=gl_accounts --table=posting_rules --table=posting_rule_lines --table=document_numbering --table=commission_rates --table=tax_codes --table=period_close_checklist --table=bank_statement_formats --table=bank_transaction_types --table=bank_match_rules` into the backup bucket.

3 Backup design

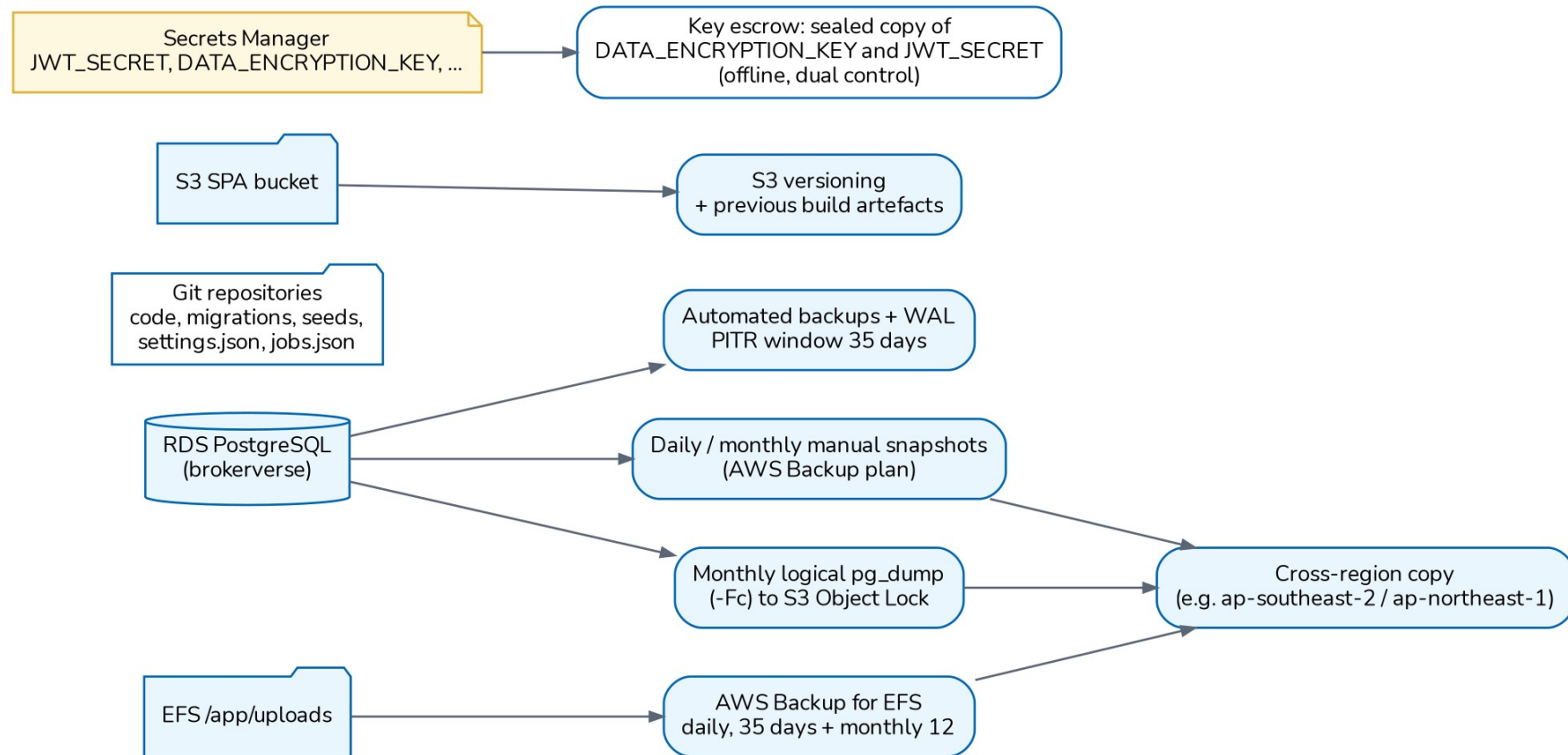


Figure 1. Backup and recovery data flows (recommended)

3.1 AWS production deployment

Backup	Configuration (Recommended / to be confirmed by the business and DevOps)	Retention
RDS automated backups and PITR	Enabled, backup window 18:00-19:00 UTC (02:00-03:00 Manila), encrypted with KMS, copy tags to snapshots, deletion protection on	35 days (PITR to any second)
RDS snapshots (AWS Backup plan)	Daily and monthly snapshots; monthly copied to a second region (for example ap-southeast-2 or ap-northeast-1); vault lock against deletion	Daily 35 days; monthly 12 months; yearly 10 years for year-end records (to be confirmed with compliance)
Pre-change snapshots	Manual snapshot before every production release with migrations, before <code>purge:sample</code> , before the go-live imports (opening balances, open items, policies) and bulk imports, and before each year-end close	Until the change is accepted (minimum 30 days)
Logical dump	Monthly <code>pg_dump -Fc</code> of the database to an S3 bucket with Object Lock (compliance mode), independent of RDS snapshots; also enables restore into any PostgreSQL 16	12 months (Glacier after 30 days)
EFS (<code>UPLOAD_DIR</code>)	AWS Backup daily; copy to the second region weekly	Daily 35 days; monthly 12 months
S3 front-end bucket	Versioning on; lifecycle removing non-current versions after 90 days	90 days
Secrets	Secrets Manager (versioned, KMS); escrow copy of <code>DATA_ENCRYPTION_KEY</code> and <code>JWT_SECRET</code> in a sealed offline store under dual control (for example two custodians)	Current and previous versions
ECR images	Lifecycle policy keeping the last 10 release tags; replication to the recovery region	10 releases

3.2 Docker Compose and single-server installations

Backup	Procedure
Database	Daily <code>docker compose exec -T db pg_dump -U brokerverse -Fc brokerverse > brokerverse-\$(date +%F).dump</code> , copied off the host (object storage), kept 35 days plus monthly copies. For PITR on a self-managed server use WAL archiving (for example pgBackRest or WAL-G).
Uploads	Daily archive of the uploads volume (<code>docker run --rm -v <project>_uploads:/data -v \$PWD:/backup alpine tar czf /backup/uploads-\$(date +%F).tgz -C /data .</code>), copied off the host.
Secrets	Keep the environment file with <code>JWT_SECRET</code> , <code>DATA_ENCRYPTION_KEY</code> , <code>DB_PASSWORD</code> in a password vault, never in the repository.

3.3 Consistency between database and files

The database and the file store are backed up separately, so a restore can yield records that point to files created after the file backup (missing files) or files without a record (orphans). Keep the EFS backup time close to the database backup window, and after a restore run the reconciliation query in the procedure below. A file referenced by a record but missing is re-uploaded by the business; generated files (reports, statements) are regenerated.

4 Restore procedures

4.1 Database: point-in-time restore (AWS)

1. Declare the incident, stop user traffic or set the application to maintenance (scale the API service to 0 containers or return a maintenance page from CloudFront) to prevent further writes to the damaged database.
2. Determine the recovery time: the last good moment before the incident (from the audit log `audit_log.at`, application logs or the user report). Record it in the incident log.
3. In RDS, "Restore to point in time" of the production instance to a new instance (same class, Multi-AZ, subnet group, security group, parameter group, KMS key) at the recovery time.
4. When the new instance is available, connect with `psql` and verify: `SELECT max(at) FROM audit_log;` is just before the recovery time; row counts of `policies`, `receipts`, `journal_vouchers` are plausible; `SELECT count(*) FROM schema_migrations;` equals the number of migration files of the deployed release.
5. Update the `DATABASE_URL` secret to the new endpoint (or rename the instances so the new one takes the old endpoint name).
6. Start the API service (or restart the containers). Each container checks `GET /api/health = 200` (database reachable, no pending migrations) before it receives traffic.
7. Smoke test: sign in, open a recent policy and its documents, run the trial balance, check Master > Schedules.
8. Inform the business of the recovery point; transactions entered after it are re-entered from source documents and the audit trail of the damaged database (keep the damaged instance, renamed, until reconciliation is complete).
9. Delete the old instance after the retention agreed in the incident review, taking a final snapshot.

4.2 Database: restore from snapshot or logical dump

1. Snapshot: "Restore snapshot" to a new instance, then steps 4 to 9 above.
2. Logical dump (other account, region or PostgreSQL server): create an empty database and a login with rights to create tables, then `pg_restore --no-owner --role=<login> -d <database> brokerverse-<date>.dump`; verify as in step 4; point `DATABASE_URL` at it and start the API.
3. The API applies any migration of a newer release on start; restoring a dump into an older release is not supported (migrations are forward only).

4.3 Files (EFS)

1. In AWS Backup, choose the EFS recovery point and restore either the whole file system to a new file system, or selected directories (for example `claim/`, `reports/`) to a new directory of the existing file system.
2. For a full restore, update the container definition to mount the new file system at `/app/uploads` and redeploy; for a partial restore, move the recovered files into place (paths are the storage keys).
3. Reconcile: list document keys missing on disk with a query such as `SELECT storage_key FROM documents WHERE status = 'uploaded'` compared with the file list (`find /app/uploads -type f`); report missing files to the business.

4.4 Front end

1. Re-run the GitHub Actions workflow on the last good commit of `dev`, or restore the previous object versions in the S3 bucket.
2. Create a CloudFront invalidation for `/*`.

4.5 Secrets

1. Restore the previous secret version in Secrets Manager (or recreate it from the escrow copy under dual control).
2. Restart the API containers so they read the secret.
3. If `PII_ENCRYPTION_KEY` is lost, the encrypted identifiers cannot be recovered: restore the key from the escrow copy; never generate a new key for an existing database except through the rotation procedure (`npm run pii:rotate,deploy/REFERENCE.md`).
4. If `DATA_ENCRYPTION_KEY` is lost permanently: generate a new key; administrators turn two-factor off for the affected users in User Management; the users enrol again. If `JWT_SECRET` is changed: users sign in again; quotation approval links and report download links already sent must be re-sent.

4.6 Application release rollback

1. Front end: as above. Backend: redeploy the previous image tag.
2. If the failed release applied migrations, the older image runs on the newer schema only if the migrations were additive (the project convention). Otherwise restore the pre-release snapshot (data entered since the release is lost and must be re-entered).

5 Restore testing schedule

Test	Frequency (recommended)	Success criteria
RDS PITR to a new instance, API started against it in an isolated UAT environment, smoke test	Before go-live, then quarterly	Restore completed within the RTO; recovery point within the RPO; smoke test passed
Logical dump restore into a clean PostgreSQL 16	Twice a year	<code>pg_restore</code> without errors; row counts equal to the source at dump time
EFS directory restore and document reconciliation	Quarterly	Sample of documents opens in the application
Secret escrow check (open the sealed copy under dual control, compare hash with Secrets Manager)	Yearly	Hashes match
Full disaster recovery exercise in the recovery region	Yearly (tabletop) / every two years (full)	Achieved RTO / RPO recorded against document 08

Each test is recorded (date, operator, backup used, recovery point, duration, issues) and reviewed by the DevOps lead; failures are raised as incidents.

6 Related documents

This document is one of eleven that together form the BrokerVerse OOTB Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery (this document)	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

7 Glossary

Terms and abbreviations used in this document.

Term	Meaning
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
Audit log / audit trail	Table audit_log: who changed which record, when, from which IP, with the before and after values.
AZ	Availability Zone: an isolated data-centre group inside an AWS region.
CloudFront	AWS content delivery network that serves the single-page application and can route /api to the backend.
CORS	Cross-Origin Resource Sharing: browser rule set that the API restricts to the configured web origins (CORS_ORIGINS).
ECR	Elastic Container Registry (AWS): stores the backend container images.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
Housekeeping	Daily job that deletes operational rows (job runs, sent e-mails, sign-in history, expired tokens, read notifications) after the retention days in System Settings.
JWT	JSON Web Token: signed token carrying the user id, roles, permissions and token version.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Multi-AZ	Deployment with a synchronous standby in a second Availability Zone and automatic failover (RDS).
PITR	Point-in-time recovery: restoring a database to any second inside the backup retention window.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in SMTP_URL.
TOTP / 2FA	Time-based one-time password (RFC 6238) used for two-factor authentication.
UAT	User acceptance testing environment.
WAL	Write-ahead log: PostgreSQL change log used for replication and point-in-time recovery.
Year-end close	Run (YEC-) that posts the closing entries in period 13, carries the balances into the next year and locks the year.